

NORTH SOUTH UNIVERSITY

Department of Electrical & Computer Engineering

PROJECT REPORT

Scientific Calculator Implementation in C

Course: CSE 115 (Introduction to Programming Language)

Section: 05

Group No: 09

Instructor: Mohammad Shifat-E-Rabbi (MSRb)

Submitted By:

Safayat Arefeen (2612096042)

Sabbir Ahmed (2611902042)

Samin Yasar Ahmed (2612524042)

Samiul Islam Sami (2612884642)

Submission Date: 30 April, 2026

2. ABSTRACT

This project presents a comprehensive command-line Scientific Calculator authored in C, developed as part of the CSE 115 curriculum. It serves as an advanced computational tool designed to handle basic arithmetic, trigonometric evaluations, matrix algebra, and number system conversions. By utilizing structured programming paradigms, the application ensures high numerical precision and features a robust input validation system (`getInt/getDouble`) to manage user errors. The implementation emphasizes modularity, allowing for efficient calculation history tracking and complex algorithm execution like recursive determinants for matrices up to 4x4.

3. INTRODUCTION

3.1 Project Objective

The primary objective was to develop a multi-functional calculator that integrates core C programming concepts—such as recursion, multidimensional arrays, and string manipulation—into a single, user-friendly interface. The project focuses on bridging theoretical mathematical logic with executable machine code while maintaining high data integrity.

3.2 Significance

In the context of NSU's CSE 115 course, this project demonstrates mastery over procedural programming. Implementing scientific features like recursive determinants and custom base conversions requires a deep understanding of memory management and standard library integration (`math.h`, `string.h`), which are critical skills for any aspiring software engineer.

4. FEATURES AND IMPLEMENTATION

4.1 Robust Input Handling (getInt/getDouble)

Directly from our source code, we implemented custom input handlers to prevent the common "infinite loop" bug in C. These functions ensure that only valid numeric data types are processed.

Snippet 4.1: Custom Input Validation

```
while(scanf("%d",&x)!=1){ printf("Invalid input: "); while(getchar()!='\n');  
// Clears the buffer }
```

4.2 Trigonometric Engine

Implemented in the `trigonometryFunction()`, the program calculates Sine, Cosine, and Tangent values. It performs an internal conversion from degrees to radians using a high-precision `PI` constant.

4.3 Base Conversion System

The `numberSystemFunction()` handles conversions between Binary, Decimal, Octal, and Hexadecimal. It utilizes a `baseToDecimal()` utility to parse strings into integers before re-encoding them into the requested base format.

4.4 Recursive Matrix Algebra

Our implementation supports matrices up to 4x4. The standout feature is the `determinant()` function, which uses recursion to solve square matrices by calculating cofactors dynamically—a significant jump in logic over basic array manipulation.

Snippet 4.2: Recursive Determinant

```
int determinant(int matrix[MAX][MAX], int n){ if(n==1) return matrix[0][0];  
if(n==2) return matrix[0][0]*matrix[1][1]-matrix[0][1]*matrix[1][0]; //  
Recursive expansion logic det+=sign*matrix[0][x]*determinant(temp,n-1); }
```

4.5 Persistent Calculation History

The program maintains a global `history` array of size 200. Using `sprintf`, each operation is logged as a descriptive string, allowing users to review their entire session via `showHistory()`.

4.6 Additional Scientific Utilities

The `scientificMode()` also includes a quadratic polynomial solver (finding roots based on the discriminant), natural and base-10 logarithms, and complex number addition.

5. IMPLEMENTATION OF OOP CONCEPTS

Although C is a procedural language, we integrated architectural patterns that mimic Object-Oriented philosophies to keep the code scalable.

5.1 Encapsulation

We grouped mathematical logic into dedicated modules like ``matrixFunction()`` and ``polynomialFunction()``. This ensures that the user interface logic is decoupled from the heavy computational engine.

5.2 Abstraction

The use of our `getInt()` and `getDouble()` wrappers abstracts the complexities of C's standard I/O and error trapping. The user sees a simple prompt, while the background logic handles potential crashes and buffer mismatches.

5.3 Modularity

The code is highly modular; each feature is an independent function. This allowed our group members to work on separate mathematical domains (e.g., Matrix vs Basic) without code overlaps, simplifying the final integration process.

6. MEMBER CONTRIBUTION

The following table outlines the distribution of tasks as implemented in the final code base for Group 09.

Member Name & ID	Features Implemented	Technical Description
Safayat Arefeen (2612096042)	Matrix, Number System, Trig, Polynomial, Core Engine	Designed the recursive determinant algorithm, cross-base conversion logic, and the overall system architecture.
Sabbir Ahmed (2611902042)	Complex Number, Sqrt, Modulus, Power Function	Authored specific scientific utilities and handled edge cases like negative square roots.
Samiul Islam Sami (2612884642)	Log, Basic Arithmetic	Developed the core arithmetic engine and the logarithmic precision modules.
Samin Yasar Ahmed (2612524042)	Basic Mode, Polynomial Logic	Built the menu interface for basic mode and shared logic for the polynomial root solver.

7. TESTING AND RESULTS

The application was subjected to various test cases to ensure mathematical accuracy.

7.1 Sample Test Scenarios

- **Quadratic Logic:** $a=1$, $b=-5$, $c=6$ resulted in Roots: 3.00, 2.00. (Passed)
- **Determinant Check:** Inputting a 3x3 matrix resulted in the correct integer output via cofactor expansion. (Passed)
- **Base Conversion:** Decimal 10 correctly converted to Binary 1010 and Hex A. (Passed)

8. CHALLENGES AND SOLUTIONS

8.1 Input Buffer Overflows

Issue: Invalid characters left in the buffer caused `scanf` to fail repeatedly.

Solution: Implemented a `while(getchar() != '\n');` loop within the `getInt()` function to flush the buffer immediately upon error.

8.2 Matrix Recursion Limits

Issue: Handling determinants of larger matrices requires complex memory state management.

Solution: We optimized the recursive calls to handle up to 4x4 matrices effectively, which covers the typical scope of undergraduate scientific calculations.

9. CONCLUSION

9.1 Summary

The "Scientific Calculator in C" successfully fulfills all course requirements by providing a stable and accurate platform for complex math. It demonstrates how procedural logic can solve real-world engineering problems through modular design.

9.2 Key Learnings

- Advanced use of `math.h` and `sprintf` for history tracking.
- Implementing recursive logic to solve multi-dimensional problems.
- The vital importance of custom input validation in professional software development.

9.3 Future Enhancements

Possible future improvements include a Graphical User Interface (GUI) and persistent file handling to save calculation history between program restarts.